Institution	SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Assessment	Assessment Tool 1 – Prompt Engineering Challenge
Name	Joanathan Packia Singh
Reg. No.	192472229
Course Code	CSA6502
Course Title	Generative AI and Large Language Models

Setup

```
!pip install -q groq
```

```
from google.colab import userdata
from groq import Groq

api_key = userdata.get('groq')
client = Groq(api_key=api_key)
MODEL = "openai/gpt-oss-120b"

def ask(prompt, system="You are a helpful assistant."):
    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": prompt}
        ]
    )
    return response.choices[0].message.content
```

Lab 6: Prompt Strategy Design

– Implement and compare the effectiveness of Zero-shot, One-shot, and Few-shot prompts.

```
zero_shot_prompt = "Classify the sentiment of this review as Positive, Negative, or Neutral: 'The delivery was late but the produ

zero_shot_output = ask(zero_shot_prompt)
print("Zero-shot Output:\n", zero_shot_output)
```

```
Zero-shot Output:
 Neutral
```

```
one_shot_prompt = '''Classify the sentiment as Positive, Negative, or Neutral.

Review: "The staff was rude and the food was cold."
Sentiment: Negative

Review: "The delivery was late but the product quality is excellent."
Sentiment:'''

one_shot_output = ask(one_shot_prompt)
print("One-shot Output:\n", one_shot_output)
```

```
One-shot Output:
 Positive
```

```
few_shot_prompt = '''Classify the sentiment as Positive, Negative, or Neutral.

Review: "The staff was rude and the food was cold."
Sentiment: Negative

Review: "Amazing experience, will come back again!"
Sentiment: Positive

Review: "The room was okay, nothing special."
Sentiment: Neutral
```

```
Review: "The delivery was late but the product quality is excellent."
Sentiment:'''

few_shot_output = ask(few_shot_prompt)
print("Few-shot Output:\n", few_shot_output)
```

```
Few-shot Output:
 Sentiment: Positive
```

```
import pandas as pd

comparison = pd.DataFrame({
    "Strategy": ["Zero-shot", "One-shot", "Few-shot"],
    "Output": [zero_shot_output.strip(), one_shot_output.strip(), few_shot_output.strip()]
})
comparison
```

	Strategy	Output
0	Zero-shot	Neutral
1	One-shot	Positive
2	Few-shot	Sentiment: Positive

Next steps: Generate code with comparison | New interactive sheet

Lab 7: Functional Prompt Development

– Design specialized prompts for summarization, email creation, and content generation.

```
text_to_summarize = '''Large language models (LLMs) are AI systems trained on massive text datasets
to understand and generate human-like language. They power applications such as
chatbots, translation tools, and content generators. Prompt engineering is the
practice of crafting effective inputs to guide these models toward desired outputs.'''

summarization_prompt = f"Summarize the following text in 2 sentences:\n\n{text_to_summarize}"
summary_output = ask(summarization_prompt)
print("Summary:\n", summary_output)
```

```
Summary:
 Large language models (LLMs) are AI systems trained on massive text corpora that can understand and generate human-like languag
```

```
email_prompt = '''Write a professional email to a professor requesting a one-day extension
for a Generative AI assignment due to a medical issue. Keep it concise and polite.'''

email_output = ask(email_prompt)
print("Email:\n", email_output)
```

```
Email:
 Subject: Request for One-Day Extension on Generative AI Assignment

Dear Professor [Last Name],

I hope you are well. I am writing to request a brief extension for the Generative AI assignment originally due on **[original du

If possible, I would greatly appreciate an additional day, moving the deadline to **[new proposed date]**. I am confident I can

Thank you for considering my request. I apologize for any inconvenience and am happy to provide documentation if needed.

Sincerely,
[Your Full Name]
[Course Title & Section]
[Student ID]
[Contact Information]
```

```
content_prompt = "Write a short, engaging paragraph introducing the topic of Generative AI for a college blog post."

content_output = ask(content_prompt)
print("Content:\n", content_output)
```

```
Content:
 Imagine a world where a computer can spin up a fresh melody, draft a research-paper abstract, or even paint a surreal landscape
```

Lab 8: API Integration

– Connect applications to OpenAI, Google Gemini, or Hugging Face Inference APIs.

```
def query_groq(prompt, model=MODEL, temperature=0.7):
    response = client.chat.completions.create(
        model=model,
        temperature=temperature,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

api_test_output = query_groq("In one line, explain what an API is.")
print(api_test_output)
```

```
An API (Application Programming Interface) is a set of defined rules and protocols that lets one software application communicat
```

Lab 9: Structured Output Generation

– Generate valid Python code and SQL queries through structured prompting techniques.

```
python_prompt = '''Generate only valid Python code, no explanation, no markdown formatting.
Write a function that takes a list of numbers and returns the average.'''

python_code_output = ask(python_prompt)
print(python_code_output)
```

```
def average(numbers):
    """Return the arithmetic mean of a list of numbers. Returns None for an empty list."""
    if not numbers:
        return None
    return sum(numbers) / len(numbers)
```

```
exec_globals = {}
try:
    exec(python_code_output.replace("```python", "").replace("```", ""), exec_globals)
    avg_func = [v for k, v in exec_globals.items() if callable(v)][0]
    print(avg_func([10, 20, 30]))
except Exception as e:
    print("Execution error:", e)
```

```
20.0
```

```
sql_prompt = '''Generate only a valid SQL query, no explanation, no markdown formatting.
Table: students(id, name, marks, department)
Query: Get the names and marks of students in the "CSE" department scoring above 80, ordered by marks descending.'''

sql_output = ask(sql_prompt)
print(sql_output)
```

```
SELECT name, marks FROM students WHERE department = 'CSE' AND marks > 80 ORDER BY marks DESC;
```

Lab 10: Strategy Evaluation

– Perform a comparative analysis of LLM responses across diverse prompting methodologies.

```
eval_prompt_base = "Explain overfitting in machine learning"

eval_zero = ask(eval_prompt_base)

eval_role = ask(f"You are an expert ML professor. {eval_prompt_base} to a first-year student in 3 sentences.")

eval_cot = ask(f"{eval_prompt_base}. Think step by step before giving the final explanation.")

eval_df = pd.DataFrame({
    "Methodology": ["Zero-shot", "Role-based", "Chain-of-Thought"],
    "Response Length (chars)": [len(eval_zero), len(eval_role), len(eval_cot)],
```

```
    "Response": [eval_zero, eval_role, eval_cot]
})
eval_df
```

	Methodology	Response Length (chars)	Response
0	Zero-shot	9481	## Overfitting in Machine Learning – A Clear, ...
1	Role-based	487	Overfitting happens when a model learns the tr...
2	Chain-of-Thought	4827	**Step-by-step thinking**\n\n1. **What is a mo...

Next steps: Generate code with `eval_df` New interactive sheet

```
for i, row in eval_df.iterrows():
    print(f"--- {row['Methodology']} ---")
    print(row['Response'])
    print()
```

```
### 10. Take-away summary

1. **Overfitting = model memorizes noise**, not just the true pattern.
2. **Detect it** by comparing training vs. validation performance, looking at learning curves, or using cross-validation.
3. **Combat it** with simpler models, regularization, more data, early stopping, dropout, ensembles, and careful feature engine
4. Think in terms of the **bias-variance trade-off**: you're always walking a tightrope between being too simple and being too

Understanding and controlling overfitting is one of the core skills that separates a "just-working" prototype from a robust, p

--- Role-based ---
Overfitting happens when a model learns the training data so well—including its random noise and quirks—that it fails to genera

--- Chain-of-Thought ---
**Step-by-step thinking**

1. **What is a model trying to do?**
   In supervised machine learning we train a model on a finite set of labeled examples (the *training set*) and we want the mod

2. **What does "fitting" mean?**
   During training the model adjusts its parameters so that its predictions match the training labels as closely as possible. T

3. **When can fitting go too far?**
   If the model becomes too closely tailored to the idiosyncrasies, noise, or outliers of the training set, it may capture patt

4. **How does overfitting manifest?**
   - **Low training error** – the model predicts the training points almost perfectly.
   - **High validation/test error** – performance drops sharply on data it has never seen.
   - The model may exhibit wildly varying predictions for inputs that are similar but not identical to training points.

5. **Why does it happen?**
   - **Model complexity**: Very flexible models (deep neural nets with many parameters, high-degree polynomials, decision tree
   - **Insufficient data**: With few training examples, the model cannot reliably infer the true underlying pattern.
   - **Noisy or mislabeled data**: The model tries to fit the noise, treating it as signal.
   - **Lack of regularization**: No constraints are placed on the magnitude or sparsity of parameters.

6. **How can we detect it?**
   - Split the data into *training* and *validation* (or use cross-validation). If validation error starts rising while trainin
   - Plot learning curves (error vs. training set size) to see a gap between training and validation performance.
   - Use statistical tests or information criteria (AIC, BIC) that penalize model complexity.

7. **How can we prevent or mitigate it?**
   - **Simplify the model** (fewer parameters, shallower networks, lower-degree polynomials).
   - **Regularization**: L1/L2 penalties, dropout (for neural nets), weight decay, early stopping, data-dependent regularizers
   - **More data**: Collect additional labeled examples or use data augmentation.
   - **Cross-validation** to choose hyper-parameters that give the best validation performance.
   - **Ensemble methods** (bagging, random forests) that average many weak learners, reducing variance.
   - **Feature selection / engineering** to remove noisy or irrelevant features.

8. **Intuition with a simple example**
   Imagine fitting a polynomial to a set of points that roughly lie on a straight line but have some random scatter.
   - A *linear* model captures the trend and predicts new points well.
   - A *10th-degree* polynomial can wiggle through every point, giving almost zero training error, but will swing wildly betwee

**Final explanation**

Overfitting occurs when a machine-learning model learns not only the underlying pattern that relates inputs to outputs, but al

The root causes are typically excessive model complexity relative to the amount (or quality) of training data, and the absence
```

Observations

- Zero-shot prompts give quick answers but with less alignment to a specific tone or level.
- Role-based prompting improves relevance and audience-appropriate depth.
- Chain-of-Thought prompting produces more structured, reasoned explanations.
- Few-shot prompting (Lab 6) improved classification consistency compared to zero-shot.